

Checklist

Platform as a Product

1. Build a Customer-Focused Product Culture

- ☐ Identify the internal teams and engineers who are your platform's customers.
- ☐ Treat internal users as **customers**, not merely stakeholders.
- ☐ Learn how customers actually work instead of relying only on what they say they want.
- ☐ Observe customer workflows, pain points, constraints, and recurring workarounds.
- ☐ Regularly interview customers to build empathy and gain insights.
- ☐ Bring customers into team meetings or all-hands to demonstrate how they use the platform.
- ☐ Have product managers regularly share customer research and feedback.
- ☐ Give engineers opportunities to participate in customer support.
- ☐ Include customer-focused goals such as adoption, satisfaction, productivity, and engagement.
- ☐ Make customer engagement a shared responsibility between product managers and engineers.
- ☐ Avoid designing only for the loudest, largest, or most influential internal team.

2. Understand the Risks of Internal Customers

- ☐ Account for having a relatively small customer base when interpreting metrics.
- ☐ Remember that internal users may be a captive audience and may have no realistic alternative.
- ☐ Do not interpret mandatory usage as proof of the product's success.
- ☐ Watch for conflicting incentives between platform teams and customer teams.
- ☐ Expect customer satisfaction to change as expectations and technology evolve.
- ☐ Assume internal engineering teams may build competing solutions if the platform does not meet their needs.
- ☐ Look beyond individual requests for patterns that apply to multiple teams.

3. Avoid the Feature-Shop Trap

- ☐ Do not build every feature customers request.
- ☐ Separate the customer's underlying problem from their proposed solution.
- ☐ Look for common needs across multiple customer groups.
- ☐ Prefer reusable platform capabilities over one-off custom implementations.
- ☐ Ask whether customers can extend or customize the platform themselves where appropriate.
- ☐ Design self-service mechanisms before demand scales.
- ☐ Consider how increased adoption will affect the platform team's workload.
- ☐ Avoid creating a growing backlog of bespoke requests that only the platform team can implement.
- ☐ Use feature requests as signals for broader product opportunities.

4. Discover Promising Platform Products

- ☐ Look for successful tools or systems already built by internal teams.
- ☐ Identify solutions that could be generalized and expanded to a wider customer base.
- ☐ Partner with a customer team to prototype new platform capabilities.
- ☐ Start with small experiments or proofs of concept before committing to a complete platform.
- ☐ Test whether a prototype solves a broader problem rather than a single team's edge case.
- ☐ Look for products with a realistic path to adoption.
- ☐ Evaluate whether customers will receive enough benefit to justify switching.
- ☐ Consider buying or adopting an existing solution instead of building when appropriate.
- ☐ Avoid copying products from large technology companies without checking whether their assumptions fit your organization.

5. Validate Product-Market Fit Internally

- ☐ Confirm that the proposed solution fits your company's technology stack.
- ☐ Identify required organizational, cultural, or process changes.
- ☐ Decide which customer segments the product is designed for.
- ☐ Estimate the number of teams or engineers who would realistically use it.
- ☐ Find potential alpha or beta customers willing to try the product.

- ☐ Validate customer demand through conversations and experiments.
- ☐ Estimate customer onboarding effort.
- ☐ Estimate migration effort from existing systems.
- ☐ Determine whether customers have the time and capacity to adopt the product soon.
- ☐ Check whether the offering is useful only for new applications or also helps existing ones.
- ☐ Test customer willingness using real commitments of time, effort, or budget rather than hypothetical enthusiasm.
- ☐ Consider the current company budget and investment climate.

6. Rethink the Problem, Not Just the Interface

- ☐ Ask whether the platform can eliminate a task rather than merely make it easier.
- ☐ Identify repetitive machine or data processes that the platform could operate centrally.
- ☐ Remove unnecessary human involvement where reliable automation is possible.
- ☐ Consider whether the platform should own the complete lifecycle of a task.
- ☐ Use “smooth the edges” improvements when coordination between people and systems is unavoidable.
- ☐ Use “rethink the problem” approaches when the task can realistically be abstracted away by a platform.
- ☐ Avoid forcing every user to maintain the same scripts, playbooks, or operational procedures independently.

7. Plan for Adoption and Migration

- ☐ Estimate the engineering cost of migration.
- ☐ Estimate the customer effort required to migrate.
- ☐ Identify compatibility problems, hidden dependencies, and legacy behaviors.
- ☐ Include training and onboarding in the migration plan.
- ☐ Provide a clear path from the old product to the new one.
- ☐ Consider whether links, integrations, workflows, and automation depend on the old system.
- ☐ Budget time for customer support during migration.
- ☐ Include migration cost when deciding whether a new product is worth building.
- ☐ Avoid launching multiple generations of the same solution without a retirement plan.

- ☐ Make adoption and migration part of the product strategy from the beginning.

8. Respect the Customer's Change Budget

- ☐ Estimate how much organizational change customers can realistically absorb this year.
- ☐ Coordinate major migrations and new tools rather than launching everything at once.
- ☐ Prioritize products that offer enough immediate benefit to justify adoption effort.
- ☐ Make new offerings as easy as possible to learn and integrate.
- ☐ Consider whether customers are already occupied with other migrations or major projects.
- ☐ Avoid assuming users will immediately adopt a new product simply because it is technically better.

9. Measure Product Success

- ☐ Define why each metric is needed before collecting it.
- ☐ Separate **impact metrics**, **guardrail metrics**, and **product health metrics**.
- ☐ Define the customer behavior or business outcome the product is meant to change.
- ☐ Track migration overhead or time imposed on users.
- ☐ Estimate time or money saved for platform customers.
- ☐ Measure voluntary adoption where possible.
- ☐ Track customer satisfaction, such as CSAT.
- ☐ Measure the user action or workflow that matters rather than only low-level system requests.
- ☐ Capture relevant user or team metadata at the time an event occurs.
- ☐ Measure outcomes across systems when the customer workflow extends beyond the platform.
- ☐ Use surveys and interviews to confirm that quantitative metrics reflect what customers value.
- ☐ Revisit metrics when they stop explaining whether the product is delivering value.
- ☐ Treat early metrics as prototypes and improve them over time.

10. Use Impact Metrics to Guide Strategy

- ☐ Write down your hypothesis about how a platform improvement causes a customer or business outcome.
- ☐ Identify intermediate signals along that cause-and-effect chain.
- ☐ Measure both the technical improvement and the resulting user behavior.
- ☐ Check whether observed results support the original hypothesis.
- ☐ Investigate alternative bottlenecks when expected outcomes do not appear.
- ☐ Adjust the strategy when evidence contradicts the impact theory.
- ☐ Use guardrails to prevent improving one metric at the expense of another.

11. Create a Product Roadmap

- ☐ Write a long-term product vision describing the desired future state.
- ☐ Identify the major obstacles preventing that vision from being achieved.
- ☐ Develop a medium-term strategy to address the most important obstacles.
- ☐ Translate the strategy into annual goals and measurable outcomes.
- ☐ Break annual goals into quarterly milestones.
- ☐ Sequence work according to impact, technical dependencies, and feasibility.
- ☐ Deliver value incrementally rather than waiting for a large final release.
- ☐ Reevaluate the roadmap as customer evidence and metrics change.
- ☐ Keep the customer-facing roadmap focused primarily on outcomes and visible capabilities rather than detailed internal engineering work.

12. Define Features by Outcomes

- ☐ Document why a feature matters to customers.
- ☐ Define the customer outcome the feature should create.
- ☐ Explain how the feature supports the larger product vision.
- ☐ Let product and engineering jointly determine the implementation.
- ☐ Break work longer than roughly a month into smaller deliverable pieces where possible.
- ☐ Avoid turning product requirements into highly prescriptive engineering specifications.
- ☐ Measure whether delivered features produced the expected customer outcome.

13. Market the Platform Internally

- ☐ Maintain an easy-to-browse internal page describing available platform products and capabilities.

- ☐ Announce meaningful launches and new features through appropriate internal channels.
- ☐ Run demos or roadshows for important platform releases.
- ☐ Build relationships with customer advocates who can evangelize useful products to their teams.
- ☐ Make sure potential users know an internal solution exists before they build or buy their own.
- ☐ Allocate explicit product-management effort to internal product marketing as the organization grows.

14. Protect Reliability and Stability

- ☐ Treat platform stability as part of the product experience.
- ☐ Prioritize reliability work when instability is damaging customer trust.
- ☐ Avoid adding new features on top of an unreliable foundation.
- ☐ Include operational quality and support costs in product decisions.
- ☐ Remember that customers may reject even valuable new products if previous offerings have been unreliable.

15. Keep Product and Engineering Responsibilities Healthy

- ☐ Ensure product managers focus on customer problems, outcomes, priorities, and strategy.
- ☐ Ensure engineering managers own engineering execution and team management.
- ☐ Do not make product managers responsible for detailed project-management work that engineering managers should own.
- ☐ Avoid treating engineering teams as implementation contractors for product specifications.
- ☐ Preserve engineers' ability to contribute ideas and shape solutions.
- ☐ Maintain enough product-management capacity to support discovery and strategy without overstaffing PMs.
- ☐ Keep product ownership distinct from backlog grooming and administrative project management.

Final Review

Before committing significant platform investment, confirm:

- ☐ We understand **who the customers are**.
- ☐ We understand **their actual problems and workflows**.
- ☐ We have evidence that the problem is worth solving.
- ☐ The solution fits our company's technical and organizational context.
- ☐ We know which customer segment we are targeting first.
- ☐ We have customers willing to test or adopt it.
- ☐ We have estimated onboarding and migration costs.
- ☐ The benefits justify the customer's effort to change.
- ☐ We have clear impact metrics.
- ☐ We have a realistic adoption strategy.
- ☐ The product supports a broader platform vision rather than isolated feature requests.
- ☐ Reliability is sufficient to earn customer trust.
- ☐ Product and engineering responsibilities are clear.
- ☐ We can explain the value of the product in terms of customer outcomes, not just technical capabilities.